Deployment Guide

This guide walks through deploying the full ThreatPulse Intel production stack on a fresh Linux server.

1. Server prerequisites

- A Linux host (Ubuntu 22.04+ / Debian 12 recommended) with **2 vCPU / 4 GB RAM** or better.
- **Docker 24+** and **Docker Compose v2**:
  ```
  bash
    curl -fsSL https://get.docker.com | sh
    docker compose version   # must print v2.x
  ```
- A **domain name** with an `A` (and optionally `AAAA`) record pointing at the server's public IP.
- Firewall allowing inbound **80** and **443**:
  ```
  bash
    sudo ufw allow 80,443/tcp
  ```

2. Clone and configure

```
git clone https://github.com/SecureAscent/threatpulse.git
cd threatpulse
make setup            # creates .env.prod
```

Edit `.env.prod` and set every value:

Variable	Notes
`DOMAIN`	Bare domain, e.g. `threatpulse.example.com`
`NEXTAUTH_URL`	`https://threatpulse.example.com` (must match `DOMAIN`)
`NEXTAUTH_SECRET`	`openssl rand -base64 32`
`POSTGRES_PASSWORD`	Strong password
`DATABASE_URL`	Must embed the SAME password
`CERTBOT_EMAIL`	Your email for expiry notices
`NVD_API_KEY`	Optional; increases NVD rate limits

> ⚠️ `DATABASE_URL` password and `POSTGRES_PASSWORD` **must match**.

3. SSL certificates

```
make ssl
```

The `init-letsencrypt.sh` script:
1. Writes your `DOMAIN` into `nginx/conf.d/threatpulse.conf` (replacing the `__DOMAIN__` token).
2. Creates a temporary self-signed cert so Nginx can boot on `:443`.
3. Starts Nginx and requests a **staging** Let's Encrypt certificate (avoids rate limits).
4. Prompts you to switch to a **production** (trusted) certificate.

If validation fails, verify DNS and that ports 80/443 are reachable from the internet.

3a. SSL for home servers / blocked port 80 (DuckDNS DNS-01)

Many home/residential connections (and CGNAT setups) **block inbound port 80**, so the HTTP challenge above will fail with `Timeout during connect (likely firewall problem)`.

If your domain is a **DuckDNS** domain (e.g. `chaugen.duckdns.org`), use the DNS-01 challenge instead — it proves ownership via a DuckDNS TXT record and needs **no open inbound ports at all**.

1. Add your DuckDNS credentials to `.env.prod`:

Variable	Notes
`DUCKDNS_TOKEN`	The token shown at the top of https://www.duckdns.org (https://www.duckdns.org) after signing in
`DUCKDNS_SUBDOMAIN`	Just the label before `.duckdns.org` — e.g. `chaugen` for `chaugen.duckdns.org`

Also make sure `DOMAIN=chaugen.duckdns.org` and `NEXTAUTH_URL=https://chaugen.duckdns.org`.

1. Run the DNS-01 flow:

```
bash
  make ssl-duckdns
```

This publishes a TXT record via the DuckDNS API, waits for propagation, obtains a **staging** cert, then prompts you to switch to a trusted **production** cert.

1. Start the stack:

```
bash
  make up
```

> **Renewal note:** the automatic `certbot` container renews via HTTP (webroot), which will not work while port 80 is blocked. On a DNS-01 setup, renew by re-running `make ssl-duckdns` (certs last 90 days). A simple cron entry keeps it current:
>
> ```bash
>
renew every 60 days at 03:30
>
> 30 3 */60 * * cd /home/user/threatpulse && yes y | make ssl-duckdns >> /var/log/threatpulse-ssl.log 2>&1
> ```

4. Launch

```
make up          # builds images and starts postgres, app, collector, nginx, certbot
make logs        # watch startup
make status      # container health
```

On first boot the `app` container:
- waits for Postgres,
- runs `prisma db push` to create tables,
- seeds demo data **only if the users table is empty**.

5. Create your admin user

```
make create-admin
# or non-interactively:
EMAIL=you@example.com PASSWORD='strongpass' ORG_NAME='Your Org' make create-admin
```

This creates a `SUPERADMIN`. Log in at `https://yourdomain.com`, then disable/delete the seeded demo accounts.

6. Certificate renewal

The `certbot` service renews automatically twice daily; Nginx reloads every 6 hours to pick up new certs. No action needed.

Updating to a new release

```
make update      # git pull + rebuild + restart + prune
```

Uninstall / reset

```
make down                                    # stop
docker compose -f docker-compose.prod.yml down -v   # stop AND delete the database
volume
```

Troubleshooting

`P1000: Authentication failed against database server`

The app/collector connect to Postgres via a `postgresql://` **URL**. If your `POSTGRES_PASSWORD` contains URL-reserved characters (`@ : / # ? $ & ! ^ %`), the URL is parsed incorrectly and authentication fails — even though the password is technically valid for Postgres itself.

Fix — use an alphanumeric-only password and reset the DB volume:

```
# 1. Stop the stack and DELETE the postgres volume (safe if you have no real
#    data yet — this wipes the database).
docker compose -f docker-compose.prod.yml down -v --remove-orphans

# 2. Generate a URL-safe password:
openssl rand -base64 48 | tr -dc 'A-Za-z0-9' | head -c 32; echo

# 3. Edit .env.prod — set BOTH to the SAME new value:
#      POSTGRES_PASSWORD=<the-new-alphanumeric-password>
#      DATABASE_URL=postgresql://threatpulse:<the-new-alphanumeric-password>@postgres:
5432/threatpulse

# 4. Start fresh:
make up
make logs        # confirm "Database is ready" then schema push succeeds
```

> **Why delete the volume?** Postgres only applies `POSTGRES_PASSWORD` when it first initializes its data directory. If you changed the password after the volume already existed, the old password is still in effect until the volume is recreated.

`Bind for :::443 failed: port is already allocated`

Another process (or a leftover container) is holding port 80/443. Find and stop it:

```
sudo lsof -i :443          # identify the process
docker ps                  # check for stray containers
docker compose -f docker-compose.prod.yml down --remove-orphans
```

Then `make up` again.